

Stereo-Vision Based 6-DoF Pose Estimation for Autonomous CubeSat Docking

Feature point:

In computer vision, a **feature point** (or keypoint) is a unique, easily identifiable location in an image that software can reliably detect and distinguish from its surroundings.

Locations where the image changes abruptly, such as **corners**, intersection points of edges, hole rims, or high-contrast patterns. A cubic satellite often has smooth, uniform sides (due to solar panels or aluminum panels). So our focus should be on **geometric corners, edges, or distinctive structural details** (such as antenna mountings, panel boundaries, or frame corners).

Camera calibration:

Since we have two Nikon cameras side-by-side (a stereo rig), we must first calibrate them using a checkerboard pattern and OpenCV's functions (`cv::stereoCalibrate` and `cv::stereoRectify`).

As a result, we can obtain the intrinsic parameters (focal length, optical center, distortion coefficients) and extrinsic parameters (the exact relative rotation and translation between the two cameras). Without this, 3D metric calculations are impossible.

CAD model:

The most powerful asset is the detailed 3D STEP model of MASAT-1. We can extract the 3D coordinates of key feature points in the satellite's local coordinate system (e.g., the 8 corners of the cube or specific docking-face vertex points). We should note the exact center of the docking face and the **outward-facing normal vector** defined in the CAD model.

Feature point detection & matching:

Locate the same feature points in both Nikon frames. Using disparity (the pixel shift between left and right views), we can compute the precise 3D position (X, Y, Z) of those points.

Once we match 4 or more detected corner/edge points in the image to their known 3D counterparts from the STEP model, we feed them into a **Perspective-n-Point (PnP)** algorithm. We need at least 4 points, that are not in the same plane. The more points we have, the better.

Pose estimation:

Using OpenCV's (`cv::solvePnP`) function in C++, you pass the 3D object points and corresponding 2D image points along with your camera matrix. The algorithm outputs the satellite's 6 Degrees of Freedom (6 DoF).

Once we have the full 6DoF pose, we can take the pre-calculated docking face normal vector from the satellite's local coordinate frame (e.g., $(0,0,-1)$). We can transform this vector and the docking center into the camera/world coordinate system using the rotation matrix and translation vector obtained from `solvePnP`.

The robot arm now receives the exact 3D coordinates of the docking center and the directional vector required for approach and alignment.

